The Journal of Open Source Software

KIM: Knowledge-Informed Mapping Toolkit

Peishi Jiang [1,2], **Aaron Wang** [1], **Susannah M. Burrows** [1], **Naser Mahfouz** [1], **and Xingyuan Chen**[1]

1 Pacific Northwest National Laboratory, Richland, Washington, USA **2** Civil, Construction and Environmental Engineering, University of Alabama, Tuscaloosa, Alabama, USA

DOI: 10.21105/joss.11112

Software

- Review
- Repository
- Archive

Editor: Wentao Ye

Reviewers:

- @lucky-verma
- @mborland

Submitted: 21 August 2025
Published: 27 October 2026

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

We present a Knowledge-Informed Mapping toolkit in the Python programming language, named KIM, to optimize the development of the mapping f from a vector of inputs $\mathbf{X}$ to a vector of outputs $\mathbf{Y}$. KIM builds on the methodology development of deep learning-based inverse mapping in Jiang et al. (2023) and Wang et al. (2026). KIM offers a preliminary understanding of data interdependencies while optimizing the training step with uncertainty accounted for. We expect this toolkit to facilitate model–data integration for Earth science applications.

Statement of need

Striving for scientific hypothesis testing and discovery, Earth scientists oftentimes develop data-driven mappings – either for inverse modeling, as part of model calibration, or forward modeling, as an emulator. Both approaches benefit from an efficient way of mapping, f, that projects from a vector of inputs $\mathbf{X}$ to a vector of outputs $\mathbf{Y}$. Such mapping approaches have seen successes in addressing inverse and forward problems in multiple studies across Earth sciences (Cromwell et al., 2021; Hu et al., 2014; Krasnopolsky & Schiller, 2003; Mudunuru et al., 2022).

Nevertheless, constructing the mapping f that connects all inputs $\mathbf{X}$ to all outputs $\mathbf{Y}$ is usually challenging due to (1) limited data/simulations for training; (2) uninformative relations between some members of $\mathbf{X}$ and $\mathbf{Y}$; and (3) the structural uncertainty of the mapping f. To address these challenges, Jiang et al. (2023) and Wang et al. (2026) leveraged the idea of integrating scientific knowledge with deep learning (Willard et al., 2022) to develop knowledge-informed mapping (KIM). The goal of this paper is to document and open source KIM for general public use. Figure 1 shows the general procedure of KIM, which is detailed in the next section.

The Journal of Open Source Software

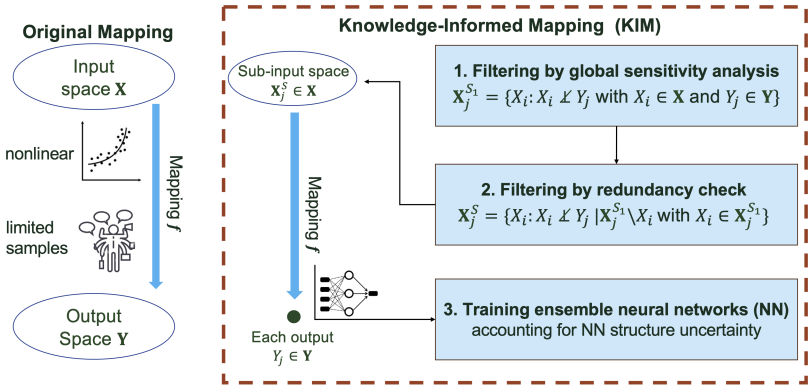


Figure 1: Comparison between KIM and the original mapping.

State of the field

Several open-source tools address individual pieces of the KIM workflow. For global sensitivity analysis, `SALib` (Herman & Usher, 2017) offers a variety of classical methods such as Sobol, Morris, and FAST. However, SALib treats sensitivity analysis as a standalone diagnostic step: it does not provide a conditional-independence-based redundancy filter, nor does it feed its results into a subsequent model-training stage. For conditional independence testing, `tigramite` (Runge et al., 2019) implements the PC algorithm for causal discovery among time series, including a conditional independence test whose residual-based implementation inspired KIM's own (Step 2 in the Mathematical approach section). However, tigramite is designed to recover causal graphs, not to construct a predictive mapping, and has no ensemble-learning or uncertainty-quantification component. For the mapping step itself, general-purpose libraries such as `scikit-learn` (Pedregosa et al., 2011) and deep learning frameworks (`JAX` (Bradbury et al., 2018), `PyTorch` (Paszke et al., 2019)) supply the regression and neural-network building blocks, but leave input pruning, ensemble configuration, and per-output uncertainty aggregation entirely to the user.

Research impact statement

KIM's contribution is to integrate these three stages, i.e., filtering by global sensitivity analysis, filtering by conditional independence, and ensemble-learning-based mapping with uncertainty quantification, into a single package that operationalizes the methodology validated in Jiang et al. (2023) and Wang et al. (2026). Rather than contributing this workflow piecemeal to SALib, tigramite, or scikit-learn, each of which serves a broader purpose, we built a focused package so the full workflow is reproducible end to end from a single dependency and openly shared Jupyter notebook examples. This mapping methodology has already been validated in peer-reviewed scientific research, rather than being a speculative or purely illustrative tool. Jiang et al. (2023), published in *Hydrology and Earth System Sciences*, applied the knowledge-informed inverse mapping approach that KIM implements to calibrate the Advanced Terrestrial Simulator against streamflow observations at the Coal Creek Watershed, Colorado – one of the two worked examples distributed with this package. Wang et al. (2026), published in *Journal of Advances in Modeling Earth Systems*, applied the same approach to estimate wall-flux and collision-kernel scaling parameters in a cloud chamber model, KIM's second worked example.

Mathematical approach

Consider a vector of inputs $\mathbf{X} = [X_1, ..., X_{N_x}]$ and a vector of outputs $\mathbf{Y} = [Y_1, ..., Y_{N_y}]$. The objective is to build up a mapping function f from $\mathbf{X}$ to $\mathbf{Y}$, such that $f : \mathbb{R}^{N_x} \rightarrow \mathbb{R}^{N_y}$, based on N_e pairs/realizations of $\mathbf{X}$ and $\mathbf{Y}$. Instead of developing a lumped mapping, we aim to develop a separate mapping f_j for each $Y_j \in \mathbf{Y}$ by using a reduced space $\mathbf{X}_j^S \subseteq \mathbf{X}$ that is most relevant to Y_j, such that $f_j : \mathbb{R}^{N_{x_j}} \rightarrow \mathbb{R}$, where $N_{x_j} = |\mathbf{X}_j^S|$ (see examples in Jiang et al. (2023) and Wang et al. (2026)), which involves the following steps.

Step 1: Filtering by global sensitivity analysis. We first perform a mutual information-based global sensitivity analysis to exclude inputs sharing zero information with Y_j and narrow down a subset $\mathbf{X}_j^{S_1}$ such that:

$$\mathbf{X}_j^{S_1} = \{X_i : I(X_i; Y_j) \neq 0 \quad \text{with } X_i \in \mathbf{X}\},$$

where $I(X_i; Y_j)$ is the mutual information between X_i and Y_j (Cover & Thomas, 2006). Based on the N_e realizations, I is calculated on the joint probability of X_i and Y_j using either the binning method or the k-nearest-neighbor method.

Step 2: Filtering by redundancy check. Then, we conduct a further assessment that filters out elements in $\mathbf{X}_j^{S_1}$ whose dynamics are redundant to Y_j given the knowledge of the remaining $\mathbf{X}_j^{S_1}$. This is achieved through a conditional independence test using conditional mutual information (Cover & Thomas, 2006) given as:

$$\mathbf{X}_j^S = \{X_i : I(X_i; Y_j | \mathbf{X}_j^{S_1} \backslash X_i) \neq 0 \quad \text{with } X_i \in \mathbf{X}_j^{S_1}\},$$

where $\mathbf{X}_j^{S_1} \backslash X_i$ is the remaining set of $\mathbf{X}_j^{S_1}$ by excluding X_i; $I(X_i; Y_j | \mathbf{X}_j^{S_1} \backslash X_i)$ is the conditional mutual information between X_i and Y_j conditioning on $\mathbf{X}_j^{S_1} \backslash X_i$. $I(X_i; Y_j | \mathbf{X}_j^{S_1} \backslash X_i) = 0$ indicates that X_i and Y_j are independent given the knowledge of $\mathbf{X}_j^{S_1} \backslash X_i$. Directly conditioning on the full $\mathbf{X}_j^{S_1} \backslash X_i$ faces the curse of dimensionality. To address this issue, we approximate it by conditioning on only the k most sensitive remaining members of $\mathbf{X}_j^{S_1} \backslash X_i$ for increasing k up to a maximum cardinality (i.e., three by default), removing X_i from $\mathbf{X}_j^S$ as soon as one such test finds independence; see the online documentation for the full algorithmic details.

Step 3: Uncertainty aware estimation by training ensemble neural networks. For each parameter Y_j, we train an ensemble of fully-connected neural networks by varying the hyperparameters, including the number of hidden layers, the number of hidden neurons, and the learning rate. We split the N_e model realizations into training, validation, and testing datasets. When evaluating the estimation on the test dataset, we further quantify the bias and uncertainty of the prediction as:

$$\begin{aligned} \text{Bias} &= E(|\mu_w - y|) \\ \text{Uncertainty} &= E(\sigma_w / |y|), \end{aligned}$$

where E is the expectation operator; y is the true value; μ_w and σ_w are the mean and standard deviation of the ensemble predictions weighted by their accuracy in the validation set. (Note that the definition of Uncertainty is relative, so it is unstable when $|y|$ is close to 0, and readers should interpret it with care.)

Software design

KIM's architecture directly mirrors the three-step methodology described above.

Two-stage filtering as a separate, model-agnostic layer. The `Data` class and the `pre_analysis` subpackage implement Steps 1-2 independently of any neural network code. Both mutual information and conditional mutual information are available via either a binning estimator or a k-nearest-neighbor estimator, and their statistical significance is assessed with a permutation shuffle test.

Two-level mapping API that encodes the modeling philosophy. The `KIM` class orchestrates one or many `Map` objects: with `map_option="many2one"`, the class trains one independent ensemble mapping per predictand Y_j, restricted to the reduced input subset selected by the chosen mask based on the filtering step; with `map_option="many2many"`, the class trains a single lumped mapping using all inputs, which serves as the non-knowledge-informed baseline used for comparison.

Ensemble uncertainty via configuration. Rather than requiring users to write a training loop per hyperparameter combination, `Map` accepts separate "choices" and "fixed" dictionaries for model, optimizer, and dataloader hyperparameters, and generates the ensemble's configurations from an `ensemble_type` of `"single"`, `"ens_random"` (random sampling), or `"ens_grid"` (full grid search). Each ensemble member is trained independently and their predictions are aggregated into a validation-loss-weighted mean and standard deviation, giving an uncertainty estimate from the ensemble itself rather than requiring a separate Bayesian or dropout-based UQ method.

Lastly, KIM reduces the computational time by adopting parallel computing in two ways. First, KIM trains ensemble members of neural networks in parallel by using `joblib.Parallel` (The joblib developers, 2025) with the `loky` backend. Second, the same technique is applied to the permutation-based shuffle tests in the sensitivity analysis. This trades inter-process overhead for training throughput that scales with available CPU cores without requiring GPU access.

Examples

We present two applications of KIM in performing inverse modeling, with Jupyter notebooks provided in the repository to guide the package usage. For each case, we developed three types of inverse mappings: (1) the original inverse mapping without knowledge-informed filtering, denoted as M_0; (2) the knowledge-informed inverse mapping only using global sensitivity analysis (Step 1), denoted as M_1; and (3) the knowledge-informed inverse mapping using both Step 1 and Step 2, denoted as M_2. The configurations can be found in the example Jupyter notebooks.

Case 1: Calibrating a cloud chamber model. Cloud chamber models have been widely applied as virtual counterparts of physical cloud chamber to study turbulence, clouds, and their interactions (Thomas et al., 2019; Wang, Ovchinnikov, Yang, Schmalfuß, et al., 2024; Wang, Ovchinnikov, Yang, Cantrell, et al., 2024; Wang, Krueger, et al., 2024; Wang et al., 2025). The objective of this example is to estimate two key parameters, i.e., the scaling coefficients of wall fluxes (λ_w) and collision processes (λ_c) using inverse mapping. To this end, an ensemble of 513 model runs were generated based on the model setup detailed in Wang et al. (2026), by varying the values of the two parameters using a Sobol sequence. Twenty-seven virtual sensors are configured, each of which 'records' multiple variables including flow properties and cloud properties. The preliminary analysis and parameter estimation results are shown in Figure 2 and Figure 3, respectively.

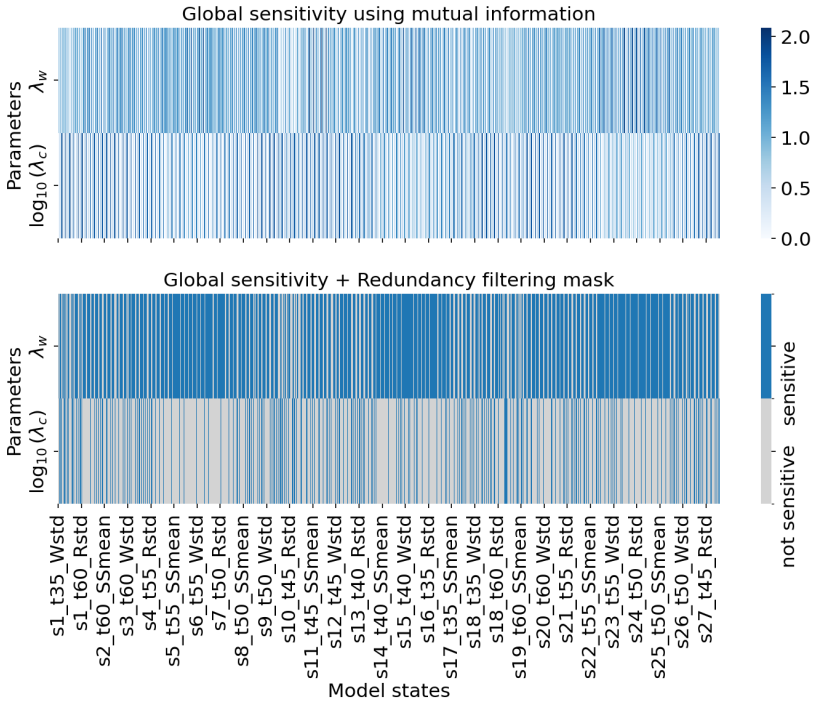


Figure 2: Preliminary analysis of cloud chamber ensemble modeling (the inputs $\mathbf{X}$: simulated flow and cloud properties at 27 sensor locations; $\mathbf{Y}$: two parameters of the cloud chamber model).

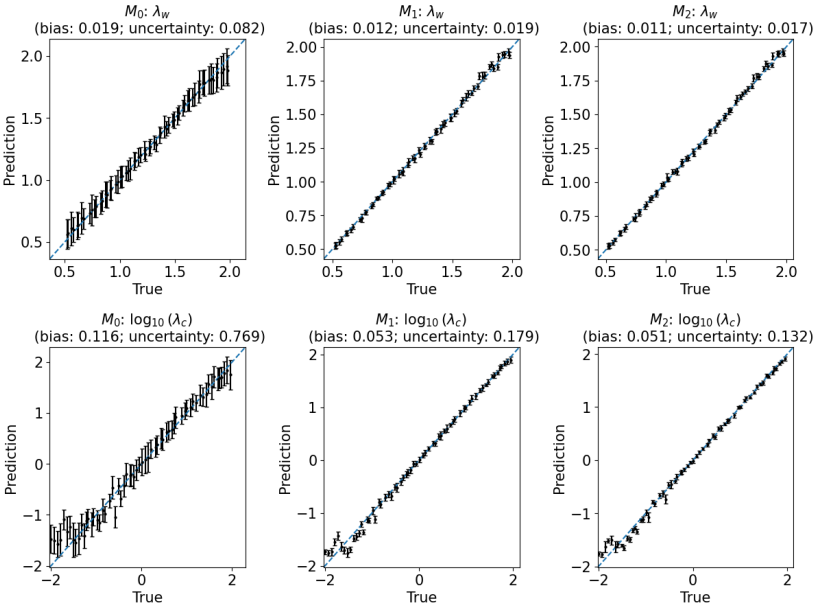


Figure 3: Estimation of the two parameters of the cloud chamber model.

Case 2: Calibrating an integrated hydrological model. The Advanced Terrestrial Simulator (ATS) is an integrated hydrological model used to simulate hydrological fluxes across a watershed (Coon et al., 2019). Here, we calibrated ATS against the streamflow observations at the outlet of Coal Creek Watershed, CO, USA. The objective is to estimate eight model parameters categorized into evapotranspiration (ET), snowmelt, and subsurface permeability.

See Jiang et al. (2023) for more detailed information. The preliminary analysis and parameter estimation results are shown in Figure 4 and Figure 5, respectively.

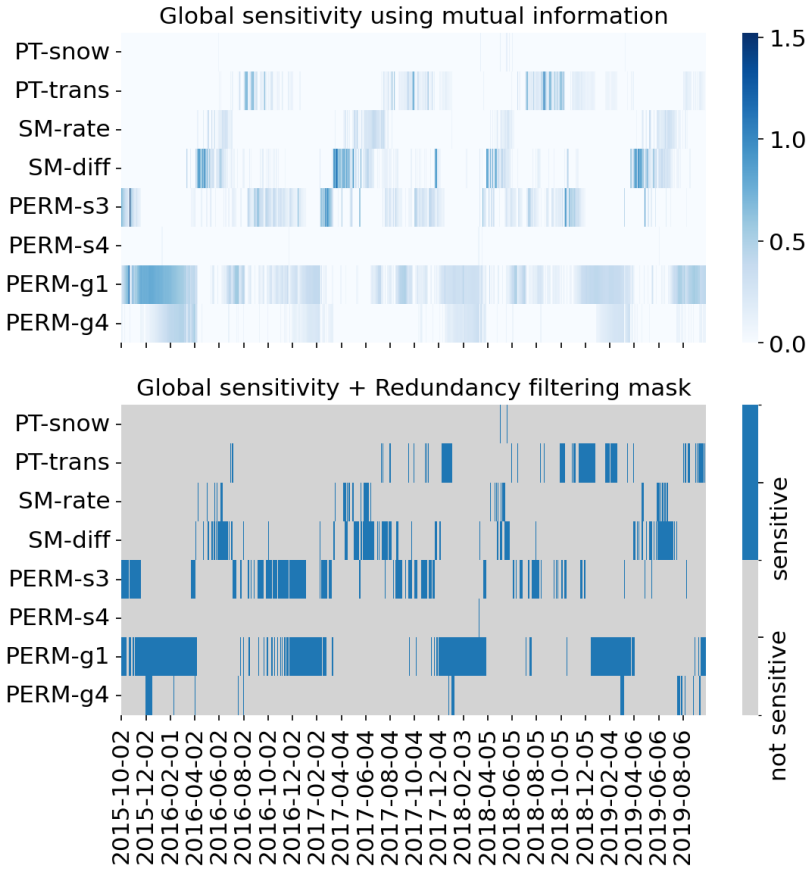


Figure 4: Preliminary analysis of ATS ensemble modeling (the inputs $\mathbf{X}$: simulated streamflows; $\mathbf{Y}$: eight parameters of the ATS model).

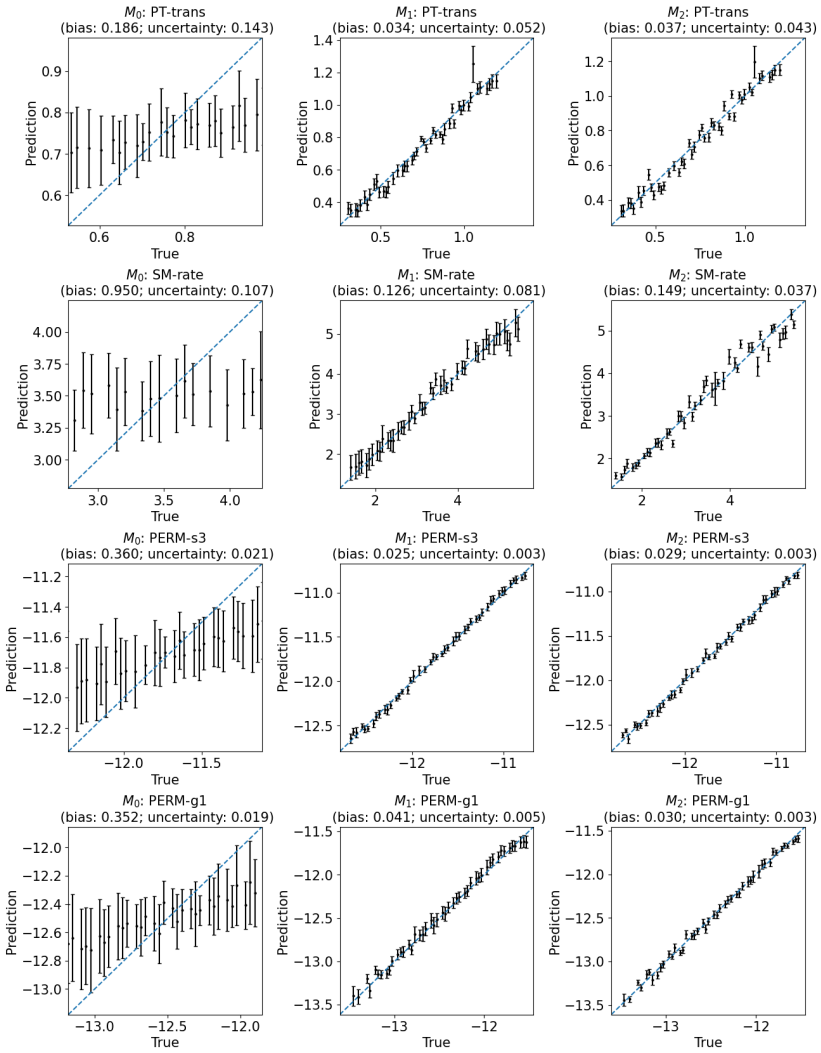


Figure 5: Estimation of four (out of eight) parameters of the ATS model.

AI usage disclosure

AI tools were used in a limited and supportive role to assist in the development of this library and in drafting the manuscript. Specifically, the authors used Claude Code (version 2.1.306 with model claude-opus-5-5) to refine the documentation and suggest code improvements. All the AI-generated code and documentation were reviewed by the authors to ensure that they accurately reflected the intended function and design of the software. The original draft of the manuscript was written by the authors and subsequently improved through feedback and revisions from the reviewers, editors, and Claude. All underlying scientific motivation, software architecture, analyses, and final interpretations were conceived, verified, and approved by the authors.

Acknowledgements

This work was supported by both the Laboratory Directed Research and Development Program at Pacific Northwest National Laboratory and the IDEAS-Watersheds project. Pacific Northwest National Laboratory is a multiprogram national laboratory operated by Battelle for the U.S.

Department of Energy under contract DE-AC05-76RL01830. The IDEAS-Watersheds project is funded by the U.S. Department of Energy (DOE), Office of Science (SC) Biological and Environmental Research (BER) program, as part of BER's Environmental System Science (ESS) program.

References

Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., & Zhang, Q. (2018). *JAX: Composable transformations of Python+NumPy programs* (Version 0.3.13). http://github.com/jax-ml/jax

Coon, E., Berndt, M., Jan, A., Svyatsky, D., Atchley, A., Kikinzon, E., Harp, D., Manzini, G., Shelef, E., Lipnikov, K., Garimella, R., Xu, C., Moulton, D., Karra, S., Painter, S., Jafarov, E., & Molins, S. (2019). *Advanced Terrestrial Simulator*. Los Alamos National Laboratory (LANL), Los Alamos, NM (United States). https://doi.org/10.11578/dc.20190911.1

Cover, T. M., & Thomas, J. A. (2006). *Elements of information theory* (2nd ed.). Wiley-Interscience. https://doi.org/10.1002/047174882X

Cromwell, E., Shuai, P., Jiang, P., Coon, E. T., Painter, S. L., Moulton, J. D., Lin, Y., & Chen, X. (2021). Estimating watershed subsurface permeability from stream discharge data using deep neural networks. *Frontiers in Earth Science*, *9*, 613011. https://doi.org/10.3389/feart.2021.613011

Herman, J., & Usher, W. (2017). SALib: An open-source Python library for sensitivity analysis. *Journal of Open Source Software*, *2*(9), 97. https://doi.org/10.21105/joss.00097

Hu, Y., Yu, X., Li, S., Chen, G., Zhou, Y., & Gao, Z. (2014). Improving the accuracy of geological model by using seismic forward and inverse techniques. *Petroleum Exploration and Development*, *41*(2), 208–216. https://doi.org/10.1016/S1876-3804(14)60024-0

Jiang, P., Shuai, P., Sun, A., Mudunuru, M. K., & Chen, X. (2023). Knowledge-informed deep learning for hydrological model calibration: An application to Coal Creek Watershed in Colorado. *Hydrology and Earth System Sciences*, *27*(14), 2621–2644. https://doi.org/10.5194/hess-27-2621-2023

Krasnopolsky, V. M., & Schiller, H. (2003). Some neural network applications in environmental sciences. Part I: Forward and inverse problems in geophysical remote measurements. *Neural Networks*, *16*(3), 321–334. https://doi.org/10.1016/S0893-6080(03)00027-3

Mudunuru, M. K., Son, K., Jiang, P., Hammond, G., & Chen, X. (2022). Scalable deep learning for watershed model calibration. *Frontiers in Earth Science*, *10*, 1026479. https://doi.org/10.3389/feart.2022.1026479

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., … Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, & R. Garnett (Eds.), *Advances in neural information processing systems* (Vol. 32). Curran Associates, Inc. https://doi.org/10.48550/arXiv.1912.01703

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., VanderPlas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, *12*, 2825–2830. https://doi.org/10.48550/arXiv.1201.0490

Runge, J., Nowack, P., Kretschmer, M., Flaxman, S., & Sejdinovic, D. (2019). Detecting and

quantifying causal associations in large nonlinear time series datasets. *Science Advances*, *5*(11), eaau4996. https://doi.org/10.1126/sciadv.aau4996

The joblib developers. (2025). *Joblib: Running Python functions as pipeline jobs*. https://github.com/joblib/joblib. https://doi.org/10.5281/zenodo.14915601

Thomas, S., Ovchinnikov, M., Yang, F., van der Voort, D., Cantrell, W., Krueger, S. K., & Shaw, R. A. (2019). Scaling of an atmospheric model to simulate turbulence and cloud microphysics in the pi chamber. *Journal of Advances in Modeling Earth Systems*, *11*(7), 1981–1994. https://doi.org/10.1029/2019MS001670

Wang, A., Jiang, P., Burrows, S. M., Glienke, S., Ovchinnikov, M., & Mahfouz, N. (2026). Inverse mapping of the collision kernel and wall flux scaling in a tall convection-cloud chamber using local sensors and knowledge-informed deep learning. *Journal of Advances in Modeling Earth Systems*, *18*(7), e2025MS005463. https://doi.org/10.1029/2025MS005463

Wang, A., Krueger, S. K., Chen, S., Ovchinnikov, M., Cantrell, W., & Shaw, R. A. (2024). Glaciation of mixed-phase clouds: Insights from bulk model and bin-microphysics large-eddy simulation informed by laboratory experiment. *Atmospheric Chemistry and Physics*, *24*(18), 10245–10260. https://doi.org/10.5194/acp-24-10245-2024

Wang, A., Ovchinnikov, M., Yang, F., Cantrell, W., Yeom, J., & Shaw, R. A. (2024). The dual nature of entrainment-mixing signatures revealed through large-eddy simulations of a convection-cloud chamber. *Journal of the Atmospheric Sciences*, *81*(12), 2017–2039. https://doi.org/10.1175/JAS-D-24-0043.1

Wang, A., Ovchinnikov, M., Yang, F., Schmalfuß, S., & Shaw, R. A. (2024). Designing a convection-cloud chamber for collision-coalescence using large-eddy simulation with bin microphysics. *Journal of Advances in Modeling Earth Systems*, *16*(1), e2023MS003734. https://doi.org/10.1029/2023MS003734

Wang, A., Schmalfuß, S., Chandrakar, K. K., Kia, H. Z., Yang, F., Ovchinnikov, M., Shaw, R. A., & Choi, Y. (2025). An intercomparison of wall fluxes in a turbulent thermal convection chamber: Direct numerical simulations and wall-modeled large-eddy simulations enhanced by machine learning. *Physics of Fluids*, *37*(4), 045163. https://doi.org/10.1063/5.0264004

Willard, J., Jia, X., Xu, S., Steinbach, M., & Kumar, V. (2022). Integrating scientific knowledge with machine learning for engineering and environmental systems. *ACM Computing Surveys*, *55*(4). https://doi.org/10.1145/3514228